



UNIVERSIDAD TECNOLÓGICA DE TECAMACHALCO

ASIGNATURA

PROGRAMACIÓN ORIENTADA A OBJETOS

MANUAL TÉCNICO DE LA APLICACIÓN GYMADMIN

Licenciatura en Ingeniería en Tecnologías de la Información e Innovación Digital

ALUMNO

OCTAVIO NOTARIO JUÁREZ

MAREYDA OREA CONTRERAS

ALEXIS GONZÁLEZ GONZÁLEZ

EDUARDO ADOLFO ROSAS ANDRADE

DOCENTE

MTRO. ELVIS COLOTLA CARVAJAL

3 "B"

TI ÁREA – DESARROLLO DE SOFTWARE MULTIPLATAFORMA

CUATRIMESTRE MAYO – AGOSTO 2026

FECHA DE ENTREGA: 17 de agosto de 2026

Índice

Introducción	3
Objetivo del sistema.....	3
Requerimientos funcionales y no funcionales	3
Arquitectura general	5
Tecnologías y dependencias.....	5
Estructura del proyecto.....	6
Modelo de base de datos.....	6
Configuración de la conexión a base de datos.....	7
Módulos del sistema	8
Roles y permisos.....	9
Capa de acceso a datos (DAO)	9
Exportación de reportes.....	10
Requisitos e instalación.....	10
Conclusión	11
Referencias bibliográficas	11

Introducción

GymAdmin (nombre interno del proyecto: GymCentral) es un sistema de escritorio desarrollado como proyecto integrador final, orientado a la administración operativa de un gimnasio. Permite gestionar clientes, pagos de mensualidades, inventario de productos y empleados, además de generar reportes exportables.

El sistema está construido en Java con interfaz gráfica JavaFX, sigue el patrón de arquitectura MVC (Modelo-Vista-Controlador) apoyado en el patrón DAO (Data Access Object) para el acceso a datos, y utiliza MySQL/MariaDB como motor de base de datos.

Objetivo del sistema

Ofrecer una herramienta centralizada que permita al personal del gimnasio:

- Registrar y consultar clientes, incluyendo el estado de vencimiento de su mensualidad.
- Registrar los pagos realizados por cada cliente.
- Llevar un inventario básico de productos en venta (suplementos, bebidas, etc.), con control manual de stock.
- Administrar empleados y sus roles de acceso al sistema.
- Generar reportes filtrables por fecha y exportarlos en formato Excel o PDF.

Requerimientos funcionales y no funcionales

A continuación se listan los requerimientos que definieron el alcance y la calidad del sistema GymAdmin, obtenidos a partir de los módulos y reglas de negocio implementados.

Requerimientos funcionales

Describen las funciones y servicios específicos que el sistema debe realizar.

ID	Requerimiento	Descripción
RF-01	Autenticación de usuarios	El sistema debe permitir iniciar sesión mediante nombre de usuario o correo electrónico junto con una contraseña, validando las credenciales contra la tabla usuario.
RF-02	Gestión de clientes	El sistema debe permitir registrar, editar, eliminar, buscar y listar clientes, mostrando el estado de vencimiento de su mensualidad.
RF-03	Gestión de pagos	El sistema debe permitir registrar, editar, eliminar y buscar pagos, asociando cada pago a un cliente existente y capturando monto, fecha de pago y fecha de vencimiento.
RF-04	Gestión de inventario	El sistema debe permitir registrar, editar, eliminar y buscar productos, controlando manualmente el stock, precio de compra y precio de venta.

ID	Requerimiento	Descripción
RF-05	Gestión de empleados y roles	El sistema debe permitir a un usuario Administrador registrar, editar, eliminar y buscar empleados, así como asignarles el rol de Administrador o Encargado.
RF-06	Control de acceso por rol	El sistema debe habilitar o deshabilitar funciones (alta, edición, eliminación, acceso a módulos) de acuerdo con el rol del usuario autenticado.
RF-07	Filtrado de información por fecha	El sistema debe permitir filtrar clientes, pagos y productos por un rango de fechas (o mostrar la totalidad si no se define un rango) desde el módulo de Reportes.
RF-08	Exportación de reportes	El sistema debe permitir exportar los datos filtrados de cada módulo (o el reporte completo) a archivos Excel (.xlsx) y PDF, permitiendo al usuario elegir la ubicación del archivo.
RF-09	Persistencia de la sesión activa	El sistema debe mantener en memoria, durante toda la ejecución, al usuario que inició sesión y su rol, para que los distintos módulos puedan consultarlo.
RF-10	Integridad referencial de la información	El sistema debe garantizar, a través de la base de datos, que al eliminar un usuario se eliminen en cascada los clientes, pagos y productos que dicho usuario haya registrado.

Requerimientos no funcionales

Describen atributos de calidad, restricciones técnicas y condiciones bajo las cuales debe operar el sistema.

ID	Requerimiento	Descripción
RNF-01	Usabilidad	La interfaz gráfica (JavaFX + CSS) debe ser clara e intuitiva, con pantallas específicas por módulo y retroalimentación visual ante errores de validación.
RNF-02	Portabilidad	El sistema debe poder ejecutarse en cualquier equipo con un JDK compatible con Java 21 o superior, sin depender de un sistema operativo específico.
RNF-03	Mantenibilidad	El código debe organizarse bajo el patrón MVC y el patrón DAO, separando entidades, acceso a datos, controladores y vistas, para facilitar futuras modificaciones.
RNF-04	Seguridad de acceso	El acceso a los módulos y operaciones críticas (edición, eliminación, gestión de empleados) debe estar restringido según el rol del usuario autenticado.
RNF-05	Disponibilidad de datos	La base de datos debe utilizar el motor InnoDB con llaves foráneas y reglas ON DELETE CASCADE / ON UPDATE CASCADE para mantener la consistencia de la información.
RNF-06	Rendimiento	Las operaciones de consulta, registro y exportación de reportes deben ejecutarse en tiempos razonables para conjuntos de datos propios de un gimnasio de tamaño pequeño o mediano.
RNF-07	Compatibilidad tecnológica	El sistema debe operar con un motor MySQL/MariaDB (recomendado mediante XAMPP) y ser gestionado mediante Apache Maven.
RNF-08	Escalabilidad de módulos	La arquitectura modular (module-info.java) debe permitir incorporar nuevos módulos o dependencias sin reestructurar el proyecto completo.

Arquitectura general

El proyecto sigue una arquitectura en capas basada en el patrón MVC, complementado con el patrón DAO para separar la lógica de acceso a la base de datos de la lógica de negocio y de presentación:

- **Modelo (Modelo):** clases de entidad (Cliente, Pago, Producto, Empleado, User) y clases DAO (ClienteDAO, PagoDAO, ProductoDAO, EmpleadoDAO, UserDAO) que ejecutan las operaciones SQL contra la base de datos.
- **Vista (resources/Vista):** archivos FXML que definen la interfaz gráfica de cada pantalla, junto con hojas de estilo CSS (resources/css).
- **Controlador (Controlador):** clases que enlazan los componentes FXML con la lógica de la aplicación (validaciones, eventos de botones, llamadas a los DAO).
- **Clase de arranque (App.Main):** inicia la aplicación JavaFX y carga la vista de Login.

Adicionalmente, la clase Modelo.SesionUsuario implementa el patrón Singleton para mantener en memoria el usuario que inició sesión y su rol durante toda la ejecución del programa.

Módulos JavaFX (module-info.java)

El proyecto está modularizado (Java Platform Module System). El módulo com.example.gymcentral declara las siguientes dependencias y aperturas relevantes:

- **requires javafx.controls, javafx.fxml, java.desktop** – interfaz gráfica.
- **requires java.sql** – conexión a la base de datos.
- **requires org.apache.poi.poi, org.apache.poi.ooxml** – exportación a Excel.
- **requires com.github.librepdf.openpdf** – exportación a PDF.
- **opens App, Controlador a javafx.fxml/javafx.graphics** – permite que JavaFX instancie los controladores.
- **opens Modelo a javafx.base** – permite que los TableView lean las propiedades de las clases de entidad.

Tecnologías y dependencias

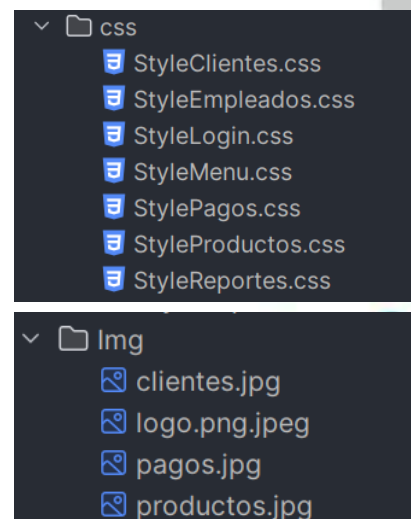
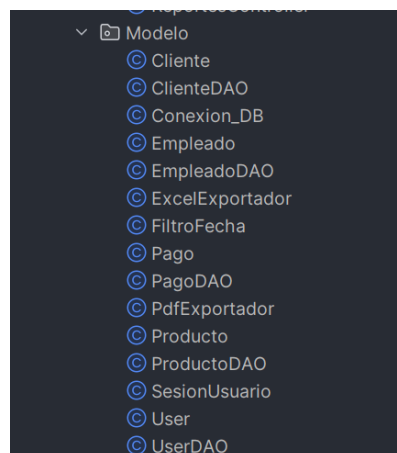
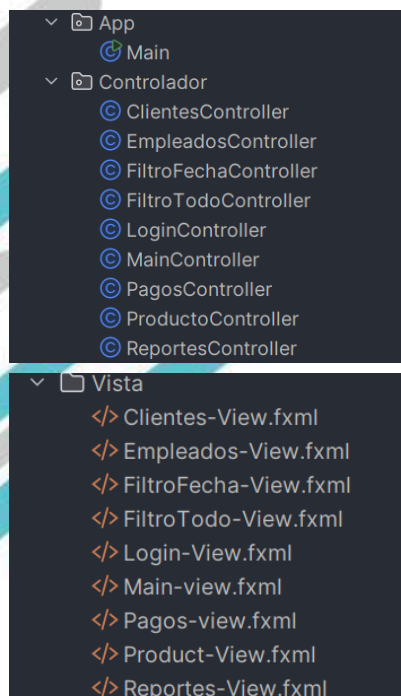
Componente	Versión	Uso en el proyecto
Java	26 (source/target en pom.xml)	Lenguaje principal
JavaFX (controls, fxml)	21.0.6	Interfaz gráfica de escritorio
MySQL Connector/J	9.0.0	Driver JDBC para conexión a MySQL/MariaDB
Apache POI (poi, poi-ooxml)	5.2.5	Generación de reportes en Excel (.xlsx)
OpenPDF	3.0.5	Generación de reportes en PDF

Componente	Versión	Uso en el proyecto
JUnit Jupiter	5.12.1 (scope test)	Pruebas unitarias (dependencia declarada)
Maven + javafx-maven-plugin	0.0.8	Gestión de dependencias y ejecución (mvn javafx:run)
MariaDB / XAMPP	10.4.32-MariaDB	Motor de base de datos, base gymadmin2

Estructura del proyecto

El proyecto se organiza en los siguientes paquetes principales:

- **App:** clase Main, punto de arranque de la aplicación JavaFX.
- **Controlador:** ClientesController, EmpleadosController, FiltroFechaController, FiltroTodoController, LoginController, MainController, PagosController, ProductoController, ReportesController.
- **Modelo:** Cliente, ClienteDAO, Conexion_DB, Empleado, EmpleadoDAO, ExcelExportador, FiltroFecha, Pago, PagoDAO, PdfExportador, Producto, ProductoDAO, SesionUsuario, User, UserDAO.
- **resources/Vista:** Clientes-View.fxml, Empleados-View.fxml, FiltroFecha-View.fxml, FiltroTodo-View.fxml, Login-View.fxml, Main-view.fxml, Pagos-view.fxml, Product-View.fxml, Reportes-View.fxml.
- **resources/css:** StyleClientes.css, StyleEmpleados.css, StyleLogin.css, StyleMenu.css, StylePagos.css, StyleProductos.css, StyleReportes.css.
- **resources/Img:** clientes.jpg, logo.png.jpeg, pagos.jpg, productos.jpg.



Modelo de base de datos

La base de datos se llama gymadmin2 y está compuesta por cuatro tablas relacionadas mediante llaves foráneas, todas ellas con motor InnoDB y borrado/actualización en cascada (ON DELETE CASCADE, ON UPDATE CASCADE) hacia usuario.

Tabla usuario

Almacena las cuentas de acceso al sistema (administradores y encargados).

Campo	Tipo	Descripción
idUsuario	int(11), PK, AUTO_INCREMENT	Identificador único del usuario
username	varchar(50)	Nombre de usuario para iniciar sesión
correo	varchar(100)	Correo electrónico (alternativa de inicio de sesión)
contrasenia	varchar(255)	Contraseña de acceso
rol	varchar(20)	Rol del usuario: Administrador o Encargado

Tabla cliente

Campo	Tipo	Descripción
idCliente	int(11), PK, AUTO_INCREMENT	Identificador único del cliente
nombre	varchar(100)	Nombre completo del cliente
telefono	varchar(15)	Teléfono de contacto
correo	varchar(100)	Correo electrónico del cliente
idUsuario	int(11), FK → usuario.idUsuario	Usuario que registró al cliente
fecha_registro	timestamp	Fecha y hora de alta del cliente (por defecto, la actual)

Tabla pago

Campo	Tipo	Descripción
idPago	int(11), PK, AUTO_INCREMENT	Identificador único del pago
monto	decimal(10,2)	Monto del pago
fechaPago	date	Fecha en que se realizó el pago
fechaVencimiento	date	Fecha en que vence la mensualidad cubierta
idCliente	int(11), FK → cliente.idCliente	Cliente al que corresponde el pago
idUsuario	int(11), FK → usuario.idUsuario	Usuario que registró el pago
fechaRegistro	datetime	Fecha y hora en que se capturó el registro

Tabla producto

Campo	Tipo	Descripción
idProducto	int(11), PK, AUTO_INCREMENT	Identificador único del producto
nombreProducto	varchar(100)	Nombre del producto
descripcion	varchar(255)	Descripción del producto (opcional)
stock	int(11)	Existencias disponibles, controladas manualmente
idUsuario	int(11), FK → usuario.idUsuario	Usuario que registró el producto
precio_compra	decimal(10,2)	Precio de compra/costo
precio_venta	decimal(10,2)	Precio de venta al público
fecha_registro	timestamp	Fecha y hora de alta del producto

Relaciones

- cliente.idUsuario → usuario.idUsuario
- pago.idCliente → cliente.idCliente
- pago.idUsuario → usuario.idUsuario
- producto.idUsuario → usuario.idUsuario

Todas las llaves foráneas están configuradas con ON DELETE CASCADE y ON UPDATE CASCADE: al eliminar un usuario se eliminan en cascada los registros de clientes, pagos y productos que él haya registrado.

Configuración de la conexión a base de datos

La clase Modelo.Conexion_DB centraliza la conexión JDBC a la base de datos gymadmin2:

```
public class Conexion_DB {
    private static final String URL = "jdbc:mysql://localhost:3306/gymadmin2"; 1 usage
    private static final String USER = "root"; 1 usage
    private static final String PASSWORD = ""; // Vacío por defecto en XAMPP 1 usage

    public static Connection getConexion() { 30 usages
        Connection conexion = null;
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            conexion = DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (ClassNotFoundException | SQLException e) {
            System.err.println("Error de conexión: " + e.getMessage());
        }
        return conexion;
    }
}
```


Módulos del sistema

Inicio de sesión (Login)

Pantalla inicial de la aplicación (Login-View.fxml, LoginController). El usuario introduce su nombre de usuario o correo y su contraseña. UserDAO.login() valida las credenciales contra la tabla usuario (permite autenticarse tanto con username como con correo) y, si son correctas, el usuario autenticado se guarda en el singleton SesiónUsuario, desde donde el resto de la aplicación consulta el rol activo.

Menú principal

Pantalla central (Main-view.fxml, MainController) con tarjetas de acceso a Clientes, Productos y Pagos, cada una mostrando el número de registros existentes, además de botones para Empleados y Reportes y un botón para cerrar sesión. Al inicializarse, el botón de Empleados se deshabilita automáticamente si el usuario activo no tiene rol Administrador.

Clientes

Gestionado por ClientesController y ClienteDAO. Permite registrar, editar, eliminar y buscar clientes, mostrando en tabla el nombre, teléfono, correo, fecha de vencimiento de la mensualidad y el usuario que registró cada cliente (registradoPor). Incluye validaciones de teléfono y correo antes de guardar.

Pagos

Gestionado por PagosController y PagoDAO. Permite registrar, editar, eliminar y buscar pagos, asociando cada pago a un cliente existente (con búsqueda de cliente por nombre o ID), capturando monto, fecha de pago y fecha de vencimiento.

Productos

Gestionado por ProductoController y ProductoDAO. Controla el inventario de productos en venta: nombre, descripción, precio de compra, precio de venta y stock (control manual, sin descuento automático por ventas). Incluye alta, edición, eliminación y búsqueda.

Empleados

Gestionado por EmpleadosController y EmpleadoDAO, accesible solo para usuarios con rol Administrador. Permite registrar, editar (con o sin cambio de contraseña), eliminar y buscar empleados, así como asignarles un rol (Administrador o Encargado) mediante un ComboBox. EmpleadoDAO.tieneMovimientosRegistrados() verifica si un empleado tiene movimientos asociados antes de permitir su eliminación.

Reportes y filtros

Gestionado por ReportesController, apoyado en FiltroFechaController, FiltroTodoController y la clase FiltroFecha. Cada módulo (Clientes, Productos, Pagos) tiene su propio filtro de rango de fechas, más un filtro

combinado (“Todo”) que aplica un rango independiente a cada entidad para un reporte conjunto. Desde esta pantalla se exportan los datos filtrados a Excel (ExcelExportador) o PDF (PdfExportador), de forma individual por módulo o como reporte completo.

Roles y permisos

El sistema define dos roles de usuario, almacenados en el campo rol de la tabla usuario y verificados en tiempo de ejecución mediante `SesionUsuario.esAdministrador()`:

Módulo / acción	Administrador	Encargado
Acceso al módulo Empleados	Sí	No (tarjeta/botón deshabilitado)
Clientes: agregar	Sí	Sí
Clientes: editar / eliminar	Sí	No (botones deshabilitados)
Pagos: agregar / guardar	Sí	Sí
Pagos: editar / eliminar	Sí	No (botones deshabilitados)
Productos: agregar / editar	Sí	Sí
Productos: eliminar	Sí	No (botón deshabilitado)

Capa de acceso a datos (DAO)

Cada entidad principal cuenta con su propia clase DAO, responsable de ejecutar las sentencias SQL mediante JDBC (PreparedStatement) usando la conexión de `Conexion_DB`.

ClienteDAO

- `obtenerTodosLosClientes()`, `obtenerClientesPorFecha(desde, hasta)`, `contarClientes()`
- `registrarCliente(Cliente)`, `modificarCliente(Cliente)`, `eliminarCliente(id)`
- `tienePagosRegistrados(idCliente)`, `buscarClientes(criterio)`

PagoDAO

- `obtenerTodosLosPagos()`, `obtenerPagosPorFecha(desde, hasta)`, `contarPagos()`
- `registrarPago(Pago)`, `modificarPago(Pago)`, `eliminarPago(id)`, `buscarPagos(criterio)`

ProductoDAO

- `obtenerTodosLosProductos()`, `obtenerProductosPorFecha(desde, hasta)`, `contarProductos()`
- `registrarProducto(Producto)`, `modificarProducto(Producto)`, `eliminarProducto(id)`, `buscarProductos(criterio)`

EmpleadoDAO

- `obtenerTodosLosEmpleados()`, `registrarEmpleado(Empleado)`

- `modificarEmpleado(Empleado)`, `modificarEmpleadoSinContraseña(Empleado)`
- `tieneMovimientosRegistrados(idUsuario)`, `eliminarEmpleado(id)`, `buscarEmpleados(criterio)`

UserDAO

- `login(identificador, password)` — autentica contra username o correo

Exportación de reportes

La generación de reportes se apoya en dos clases especializadas dentro del paquete Modelo:

ExcelExportador (Apache POI)

- `exportarTodo(clientes, productos, pagos, archivo)` — libro con una hoja por entidad
- `exportarClientes(clientes, archivo)`
- `exportarProductos(productos, archivo)`
- `exportarPagos(pagos, archivo)`

PdfExportador (OpenPDF)

- `exportarTodo(clientes, productos, pagos, archivo)` — documento PDF con las tres secciones
- `exportarClientes(clientes, archivo)`
- `exportarProductos(productos, archivo)`
- `exportarPagos(pagos, archivo)`

Desde `ReportesController`, cada exportación aplica el rango de fechas definido en el filtro correspondiente (o la totalidad de los registros si no se definió filtro) y el usuario elige la ubicación del archivo mediante un `FileChooser`.

Requisitos e instalación

Requisitos previos

- JDK compatible con Java 21+ (el proyecto compila con source/target 26).
- Apache Maven (o usar el wrapper incluido `mvnw` / `mvnw.cmd`).
- Servidor MySQL/MariaDB — se recomienda XAMPP, con el usuario root sin contraseña.

¿Cómo iniciar con la aplicación?

- 1. Iniciar el servicio de MySQL en XAMPP.
- 2. Crear la base de datos `gymadmin2` e importar el archivo `gymadmin2.sql` proporcionado (incluye las tablas usuario, cliente, pago y producto, con datos de ejemplo).

- 3. Verificar en Conexion_DB.java que la URL, el usuario y la contraseña coincidan con la configuración local.
- 4. Desde la raíz del proyecto, ejecutar: mvn clean javafx:run
- 5. Iniciar sesión con un usuario existente en la tabla usuario (por ejemplo, el usuario Magnum con rol Administrador incluido en el respaldo de la base de datos).

Conclusión

El desarrollo de GymAdmin permitió aplicar de manera integral los conceptos de la Programación Orientada a Objetos vistos durante el cuatrimestre, desde el diseño de clases de entidad y su relación mediante composición y asociación, hasta la implementación de patrones de diseño como MVC, DAO y Singleton en un proyecto de escritorio funcional.

A lo largo del desarrollo se reforzó el manejo de JavaFX para la construcción de interfaces gráficas, la conexión a bases de datos relacionales mediante JDBC, el uso de bibliotecas externas (Apache POI y OpenPDF) para la generación de reportes en distintos formatos, y la gestión de dependencias con Maven. Asimismo, la incorporación de roles de usuario (Administrador y Encargado) permitió aplicar principios básicos de control de acceso y seguridad a nivel de aplicación.

El trabajo colaborativo en equipo fue clave para dividir responsabilidades entre los módulos de Clientes, Pagos, Productos, Empleados y Reportes, manteniendo una arquitectura consistente y un mismo estándar de codificación. Como resultado, se obtuvo un sistema funcional, modular y fácil de mantener, que cumple con los requerimientos planteados y que además representa una base sólida sobre la cual podrían incorporarse mejoras futuras, tales como el descuento automático de stock por ventas, el envío de notificaciones de vencimiento de mensualidades, o la migración de la interfaz a un entorno web.

Referencias bibliográficas

- Oracle. (2024). Documentación oficial de Java SE. Oracle Corporation. <https://docs.oracle.com/en/java/javase/>
- Gluon / OpenJFX. (2024). JavaFX Documentation. <https://openjfx.io/>
- Oracle. (2024). Java Database Connectivity (JDBC) API. <https://docs.oracle.com/javase/tutorial/jdbc/>
- Apache Software Foundation. (2024). Apache POI – the Java API for Microsoft Documents. <https://poi.apache.org/>
- LibrePDF Project. (2024). OpenPDF – Free Java library for PDF files. <https://github.com/LibrePDF/OpenPDF>
- Apache Software Foundation. (2024). Apache Maven Project. <https://maven.apache.org/>
- Oracle / MySQL. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>

- MariaDB Foundation. (2024). MariaDB Server Documentation. <https://mariadb.com/kb/en/documentation/>
- Apache Friends. (2024). XAMPP – Documentación oficial. <https://www.apachefriends.org/>
- The JUnit Team. (2024). JUnit 5 User Guide. <https://junit.org/junit5/docs/current/user-guide/>